

L'approche Objet

La réalisation d'un projet informatique suit idéalement 3 phases successives :

- **Analyse** : comprendre clairement le problème posé ;
- **Conception** : identifier les concepts fondamentaux impliqués dans une solution ;
- **Programmation** : exprimer la solution dans un programme informatique.

Deux approches pour réaliser un projet informatique

1) Approche fonctionnelle (début des développements informatiques, années 1960-80) :

- Données séparées des traitements ;
- Concept clef : la fonction (procédure, routine, ...) → découpage du problème en fonctions ... découpées en sous-fonctions ...

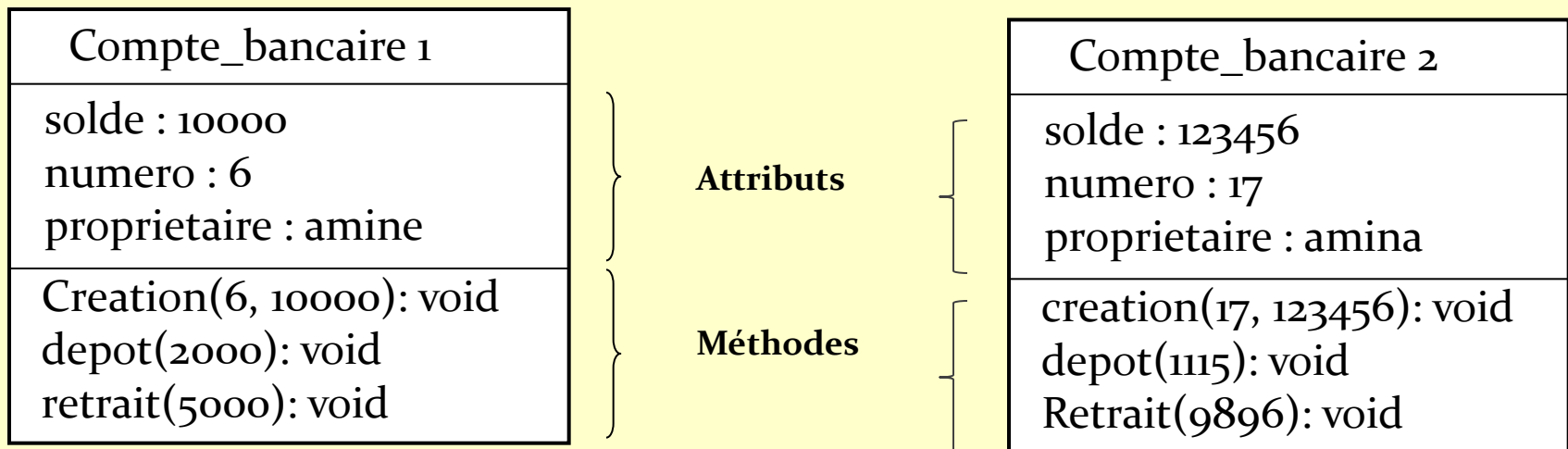
C'est le cas des programmes vus en S3 (programmation en C) et S4 (structures de données)

L'approche Objet

2) **L'approche Objet**: rapprocher les entités informatiques de celles du monde réel ...

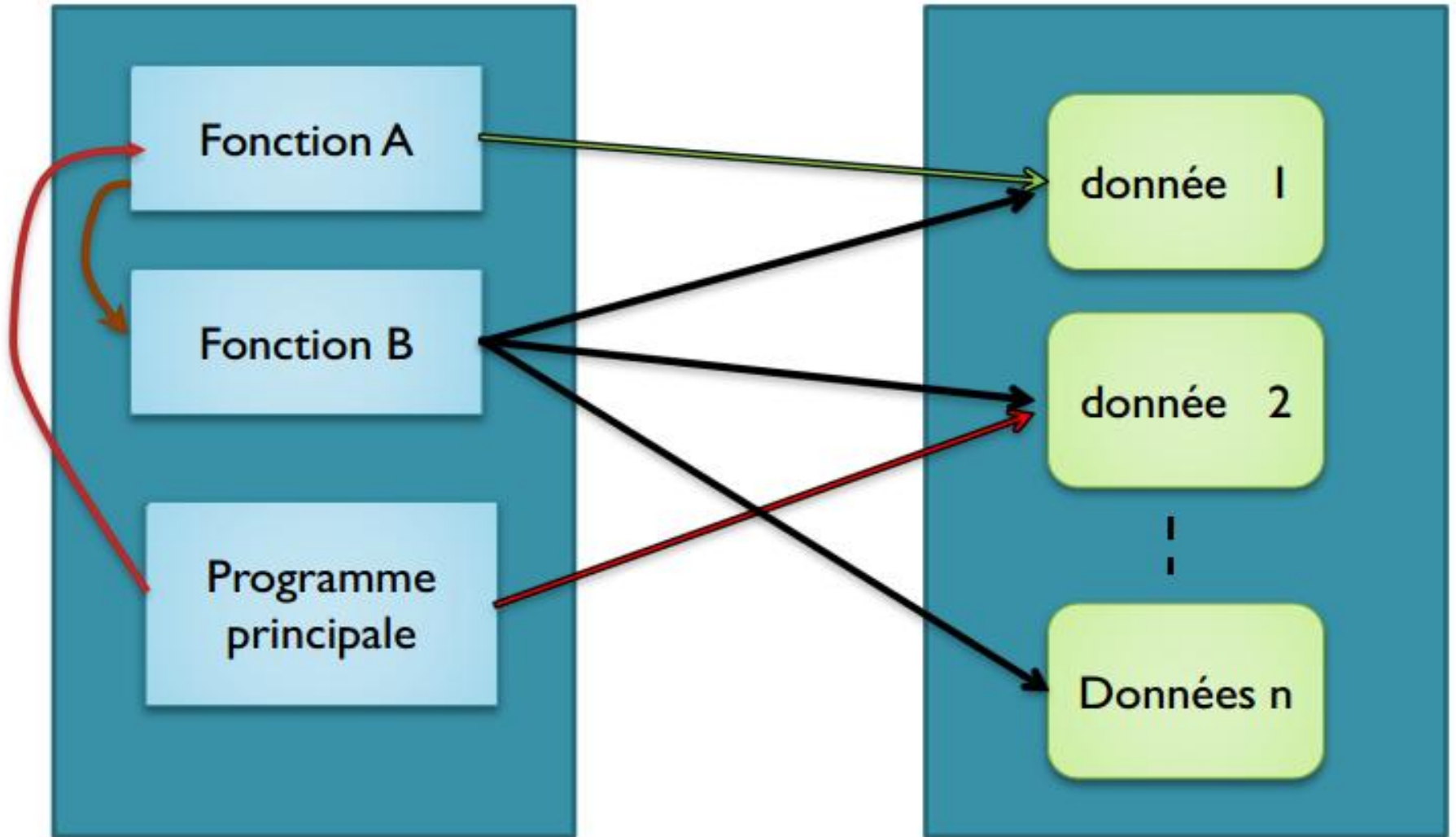
→ Faire des objets communicants entre eux possédant :

- des propriétés → données
- des comportements → méthodes



Deux objets de type Compte_bancaire

L'approche Objet



Le concept d'Objet

- ✚ Un **objet** informatique, au sens de l'**Orienté Objet**, est une unité atomique possédant :
 - une identité ;
 - un état, défini par un ensemble de données (attributs, ou données membres) ;
 - un comportement, défini par un ensemble de fonctions (méthodes, ou fonctions membres).
- ✚ Un **Objet** peut correspondre à :
 - Un objet concret du monde réel, ayant une réalité physique (une personne, une voiture, un système mécanique, **un étudiant**, . . .) ;
 - Un concept abstrait (un compte bancaire, une contrainte mécanique, une vitesse, une équation différentielle, . . .) ;
 -

Les deux principaux paradigmes de la programmation objet

- **L'abstraction**, c'est la capacité d'ignorer les détails d'un élément pour en avoir une vision globale.

L'abstraction de données consiste à regrouper des données, ce qui aboutit aux types complexes de données (comme les structures en C).

Le modèle objet est une abstraction des données et des traitements (fonctions qui seront appliquées sur ces données).

- **L'encapsulation**, c'est le masquage de certains éléments au sein d'une entité (un "module", ou un objet). Ils ne sont plus visibles (ou accessibles) à l'extérieur de l'entité.
Dans le modèle objet, un objet n'est visible de l'extérieur que par son *interface*.

Introduction à la programmation Orienté Objet sous python 3

```
1 # partie 1.py
2 # définition de la classe Compte
3 class compte:
4     """Un exemple de classe :gestion d'un compte bancaire"""
5     nombreDeComptes=0 #un attribut statique ou de classe
6     #definition du constructeur
7     def __init__(self,soldeInitial,nomClient,numCompteClient,adressCli="tanger"):
8         print("je suis le constructeur")
9         """Initialisation du compte avec la valeur soldeInitial."""
10        # assignation de l'attribut d'instance solde
11        self.__solde = soldeInitial #attribut privé d'instance
12        self.__nom=nomClient          #attribut privé d'instance
13        self.numCompte=numCompteClient #attribut publique d'instance
14        self.__class__.nombreDeComptes+=1 #attribut statique
15        self.adresse=adressCli
```

Introduction à la programmation Orienté Objet sous python 3

```
def GetSold(self):  
    return self.__solde  
  
def __del__(self):  
    """desctruction du compte """  
    print("je suis le destructeur")  
    self.__class__.nombreDeComptes-=1  
  
def affiche(self):  
    print("Le client, Mr/Mme/Mlle ",self.__nom," a: ",self.__solde,end=' ' )  
    print("dans le compte: ",self.numCompte)  
# définition de la méthode NouveauSolde()  
  
def afficherNombreDecomptes(cls): #méthode statique ou de classe  
    print("Actuellement, il y a ",__class__.nombreDeComptes," comptes cres...")
```

Introduction à la programmation Orienté Objet sous python 3

```
def NouveauSolde(self,valeurSolde):
    self.__solde = float(valeurSolde)
# définition de la méthode Solde()
def SoldeClient(self):
    return self.__solde
# définition de la méthode AjouterDansCompte()
def AjouterDansCompte(self,somme):
    self.__solde += somme
# définition de la méthode Debit()
def Debit(self,somme):
    self.__solde -= somme
# définition de la méthode spéciale __add__ (surcharge de l'opérateur +)
def __add__(self,somme):
    self.__solde += somme
# définition de la méthode spéciale __sub__ (surcharge de l'opérateur -)
def __sub__(self,somme):
    self.__solde -= somme
```

Introduction à la programmation Orienté Objet sous python 3

```
def __mul__(self, m):
    # redéfinit le comportement de l'opérateur *
    self.__solde *= m
def __str__(self):
    x="Le client, Mr/Mme/Mlle "+self.__nom+" a: "+str(self.__solde)
    x=x+" dans le compte: "+str(self.numCompte)
    return x
if __name__ == '__main__':
    y=compte(100,'rita','516465')
    y.affiche()
    print(y)
    y+20
    y.affiche()
    # taper compte. et voir les options (méthodes et membres statiques)
    compte.afficherNombreDecomptes(compte)
```

Introduction à la programmation Orienté Objet sous python 3

```
class User:    # partie 2
    pass
def test1():# nous sommes hors la classe user !!!
    Amine = User()
    ### Définition d'attributs pour cet objet
    Amine.id = 1
    Amine.name = 'Alami'
    Amine.password = '12345'
    Amine.__password = 'azerty'
    Amine.solde = 100000
    print('Bonjour, je suis '+Amine.name)
    print('Mon id est: '+str(Amine.id))
    print('Mon mot de passe est : '+str(Amine.password))
    print("+++++")
    Amine.password = 'mot de passe plus sécurisé !'
    print(Amine.password)
    del(Amine.password)
    Amine.password='Nouveau Mot de passe'
    del(Amine.__password)
    Amine.__password='même avec attribut privé!'
```

Introduction à la programmation Orienté Objet sous python 3

```
1 class User:#partie 2
2     __slots__ = {'age','name','password','niveau'}
3     def __init__(self,a=25,na='',p='',niv=''):
4         self.age=a
5         self.name=na
6         self.password=p
7         self.niveau=niv
8     pass
9
10 def test1():
11     Amine = User()
12     ### Définition d'attributs pour cet objet
13     #Amine.id = 1
14     #opération non autorisée car id n est pas dans __slots__
15     Amine.name = 'Alami'
16     Amine.password = 'azerty'
```

Introduction à la programmation Orienté Objet sous python 3

```
class Animal : # partie 3
    def __init__(self, name):
        self.name= name
    def showInfo(self):
        print ("I'm " + self.name)

# La classe Cat est étendue (extends) de la classe Animal.
class Cat (Animal):
    def __init__(self, name, age):
        super().__init__(name)
        self.age = age

    # Remplacez (override) la méthode ayant le même nom de la classe mère.
    def showInfo(self):
        print ("I'm " + self.name)
        print (" age " + str(self.age))

tom = Cat("Tom", 3)
print ("\n")
print ("Call showInfo() method:")
tom.showInfo()
Animal.showInfo(tom) # pour appeler une méthode de la classe mère sur l'objet de la classe fille
```

Introduction à la programmation Orienté Objet sous python 3

```
class Etre_vivant_vaccine_contre_virus():
    def __init__(self, Vacc_ok, VaccNonOk):
        self.vaccins_faits=Vacc_ok
        self.vaccins_Non_faits=VaccNonOk
        # La méthode (method):
    def showInfo(self):
        print ("Vaccins faits:",self.vaccins_faits)
        print ("Vaccins non faits:",self.vaccins_Non_faits)

    # La méthode (method):
    def faire_vaccin(self, vaccin):
        if vaccin in self.vaccins_Non_faits:
            self.vaccins_faits.append(vaccin)
            self.vaccins_Non_faits.remove(vaccin)
```

Introduction à la programmation Orienté Objet sous python 3

```
class Animal_vaccine_contre_virus(Animal,Etre_vivant_vaccine_contre_virus):
    def __init__(self, name, VaccOk,VaccNonOk):
        Animal.__init__(self, name)
        Etre_vivant_vaccine_contre_virus.__init__(self,VaccOk,VaccNonOk)

    def showInfo(self):
        print ("-- Call Muliclass.showInfo: --")
        Animal.showInfo(self)
        Etre_vivant_vaccine_contre_virus.showInfo(self)

c1=Animal_vaccine_contre_virus('Cat-1',['v1','v2','v3'],['v4','v5'])
c1.showInfo()
c1.faire_vaccin('v4')
c1.showInfo()
```

Surcharge d'opérateurs

Opérateur	Expression	Interprétation Python
Addition	$p1 + p2$	<code>p1.__add__(p2)</code>
Soustraction	$p1 - p2$	<code>p1.__sub__(p2)</code>
Multiplication	$p1 * p2$	<code>p1.__mul__(p2)</code>
Puissance	$p1 ** p2$	<code>p1.__pow__(p2)</code>
Division	$p1 / p2$	<code>p1.__truediv__(p2)</code>
Division entière	$p1 // p2$	<code>p1.__floordiv__(p2)</code>
le reste (modulo)	$p1 \% p2$	<code>p1.__mod__(p2)</code>

Surcharge d'opérateurs

Opérateur	Expression	Interprétation Python
Inférieur à	$p1 < p2$	<code>p1.__lt__(p2)</code>
Inférieur ou égal	$p1 \leq p2$	<code>p1.__le__(p2)</code>
Egal	$p1 == p2$	<code>p1.__eq__(p2)</code>
différent	$p1 \neq p2$	<code>p1.__ne__(p2)</code>
Supérieur à	$p1 > p2$	<code>p1.__gt__(p2)</code>
Supérieur ou égal	$p1 \geq p2$	<code>p1.__ge__(p2)</code>

Surcharge d'opérateurs

opération	symbole	méthode
opposé	-	<code>__neg__(self)</code>
positif	+	<code>__pos__(self)</code>
valeur absolue	<code>abs()</code>	<code>__abs__(self)</code>

opération	syntaxe	méthode
dimension	<code>len(objet)</code>	<code>__len__(self)</code>
accès aux éléments en lecture	<code>objet[key]</code>	<code>__getitem__(self, key)</code>
accès aux éléments en écriture	<code>objet[key]</code>	<code>__setitem__(self, key, value)</code>

Introduction à la programmation Orienté Objet sous python 3

```
1 import math # partie 4
2 class Point:
3     def __init__(self, x, y):
4         self.__x = x
5         self.__y = y
6
7     def __str__(self):
8         return "x="+str(self.__x)+" et y= " + str(self.__y)
9
10    def __add__(self, p):
11        a = self.__x + p.__x
12        b = self.__y + p.__y
13        return Point(a, b)
14
15    def __lt__(self, p):# Opérateur <
16        m_self = math.sqrt((self.__x ** 2) + (self.__y ** 2))
17        m_p = math.sqrt((p.__x ** 2) + (p.__y ** 2))
18        return m_self < m_p
19
20    def __le__(self, p):# Opérateur <=
21        m_self = math.sqrt((self.__x ** 2) + (self.__y ** 2))
22        m_p = math.sqrt((p.__x ** 2) + (p.__y ** 2))
23        return m_self <= m_p
```

Introduction à la programmation Orienté Objet sous python 3

```
43 def __getitem__(self, key):
44     if key==0:
45         return self.__x
46     elif key==1:
47         return self.__y
48     else:
49         return "attention, entrer un indice correcte"
50
51 def __setitem__(self, key, nouvelle_Valeur):
52     if key==0:
53         self.__x=nouvelle_Valeur
54     elif key==1:
55         self.__y=nouvelle_Valeur
56     else:
57         print("attention, entrer un indice correcte")
58
59 p1 = Point(2, 4)
60 p2 = Point(5, 1)
61 p3 = p1+p2
62 print(p3)
63 print(p1<p2)
64 print(p1.__ge__(p2))
```

Introduction à la programmation Orienté Objet sous python 3

```
1 class User:      # partie 5
2     def __init__(self, NumCIN, NumPermi_conduire):
3         self.__CIN=NumCIN
4         self.__NPERM=NumPermi_conduire
5     def affiche(self):
6         print("CIN=",self.__CIN)
7         print("Numéro de Permis=",self.__NPERM)
8
9 Amine = User(12345, "PERM456778")
10 #del(Amine.__CIN)
11 print(Amine.__dict__.keys())
12 Amine.affiche()
13 del(Amine.__dict__['_User__CIN'])
14 print(Amine.__dict__.keys())
15 Amine.affiche()
```

```
>>> (executing lines 1 to 17 of "_partie_5_Encapsulation_de_données.py")
dict_keys(['_User__CIN', '_User__NPERM'])
CIN= 12345
Numéro de Permis= PERM456778
dict_keys(['_User__NPERM'])
Traceback (most recent call last):
  File "C:\partie_5_Encapsulation_de_données.py", line 17, in <module>
```

Surdéfinition de fonctions

```
def f(x):  
    print("je suis une fonction avec un seul paramètre:"+str(x))  
  
def f(x,y):  
    print("je suis une fonction avec deux paramètres:"+str(x)+ '  
    et '+ str(y))  
  
print(f(100))
```

Traceback (most recent call last):
File "C:_____python\polymorphisme.py", line 7, in
<module>
print(f(100))
TypeError: f() missing 1 required positional argument: 'y'

Surdéfinition de fonctions

```
from multipledispatch import dispatch
@dispatch(object)
def f(x):
    print("fonction avec un seul paramètre:"+str(x))
@dispatch(object,object)
def f(x,y):
    print("fonction avec deux paramètres:"+str(x)+ ' et '+ str(y))
```

```
print(f(100))
print(f(1111,2222))
```

=====➔

je suis une fonction avec un seul paramètre:100

je suis une fonction avec deux paramètres:1111 et 2222